

ADDITIONAL POWER REPOSITORY: 19741vanch/OEW @ stef49d

Target MCU: STM32G4RE Field Oriented M2, 170kHz system: PWM generation, dead time insertion, ADC calibration, and readiness assessment for STEVAL-IPM20B integration.

Date: July 28, 2026

Status:

DEAD-TIME BLOCKER

1. Introduction and Project Context	2
1.1. System Architecture	2
2. Test Results Analysis	2
2.1. System Information	2
2.2. PWM Auto Test	3
2.3. Dead-Time Measurement	3
2.4. ADC Calibration	4
3. Identified Defects	4
3.1. Critical Defects	4
3.1.1. Missing _export_csv Method in SaleaeHelper	4
3.1.2. TIMER_CLK / tclk Inconsistency	5
3.1.3. Dead-Time Mismatch: 95 ns vs. 1,600 ns	5
3.2. Medium Severity Defects	5
3.2.1. Bare except Clauses	5
3.2.2. Race Condition in _tr_cache	5
3.2.3. Hardcoded Saleae Parameters	5
3.3. Low-Severity Issues	6
3.3.1. Debug Button in Production Code	6
3.3.2. FOC Data Buffer for CSV Export	6
3.3.3. Lambda Closure Fixes (Partially Addressed)	6
4. Readiness Assessment for STEVAL-IPM20B	6
5. Recommendations	7
5.1. Priority Actions (Before Connecting STEVAL)	7
5.2. Medium-Priority Actions	7
5.3. Future Improvements	8

1. Introduction and Project Context

This document presents an audit of the OEW Motor FOC project at commit 9fef49d of the 1974Ivanich/OEW repository. The project implements a Field Oriented Control (FOC) system for a three-phase brushless DC motor (BLDC/PMSM) on the STM32G474RE microcontroller (Cortex-M4 core, 170 MHz). The hardware platform consists of a Nucleo-G474RE board and a STEVAL-IPM20B driver board with integrated MOSFET transistors in a half-bridge configuration for the three-phase inverter. The primary goal of the project is to develop debugging tooling and firmware for calibrating and testing the inverter power stage.

The analysis is based on the `nucleo_debug_tool.py` source code (998 lines), the results of automated PWM signal testing via Saleae Logic 2, dead-time measurements, ADC calibration results, and the TIM1 register dump. The firmware source files (`main.c`, `pwm.c`, `adc.c`, `system_stm32g4xx.c`) were not available for direct review due to the private nature of the repository; conclusions about firmware behavior are derived exclusively from telemetry data transmitted over UART.

1.1. System Architecture

The system comprises three main components. First, the STM32G474RE firmware, which configures clock generation from the HSE crystal through the PLL (core frequency 170 MHz, TIM1 TCLK = 10 MHz at PSC=16), initializes UART2 at 115200 baud for command and telemetry exchange, configures TIM1 in centre-aligned PWM mode (CMS=11, CR1=0xC0) for generating three pairs of complementary PWM signals with programmable dead-time (BDTR register), and sets up ADC2 for injected conversions of four channels (I1, I2, IN, VBUS) with a hardware trigger from TIM1 Update events.

Second, the Nucleo Debug Tool GUI implemented in Python/Tkinter, containing three tabs: PWM Test (PWM generation control, automated frequency and duty cycle testing via Saleae, dead-time measurement), ADC Test (ADC value reading, zero-offset calibration, data streaming), and FOC Full (FOC algorithm start/stop, speed reference, Id/Iq control). Third, the Saleae Logic 2 logic analyzer connected via gRPC Automation API for capturing and analyzing digital signals at 24 MHz sampling rate.

2. Test Results Analysis

All results were obtained with the Saleae Logic 2 connected directly to the microcontroller outputs (without STEVAL-IPM20B or motor). This means the tests verify the correctness of TIM1 PWM signal generation but do not reflect real power stage behavior under load.

2.1. System Information

The system register dump obtained via the UART "sysinfo" command shows: SYS:CLK=170000000, PSC=16, TCLK=10000000, PLLCFGR=0x01005532. The SYSCLK frequency of 170 MHz matches the expected value when running from HSE (24 MHz) through the PLL. Decoding PLLCFGR=0x01005532: PLLSRC=HSE (bit 0), PLLM=3 (bits 4:6=011), PLLN=170 (bits 8:14=10101010), PLLR=2 (bits 29:30=10), PLLQ=8 (bits 24:27=100). Calculation: $TIM_CLK = HSE/M*N/Q = 24/3*170/8 = 170\text{ MHz}$, $TCLK =$

$TIM_CLK/(PSC+1) = 170e6/(16+1) = 10,000,000 \text{ Hz (10 MHz)}$. All values match expectations with no PLL configuration error.

2.2. PWM Auto Test

Automated PWM test results for all six channels show 100% pass rate (6 PASS, 0 FAIL). The measured and expected parameters are shown in the table below.

Channel	Meas. Freq	Exp. Freq	Freq Err	Meas. Duty	Exp. Duty	Duty Err
PC0 HIN_U1	50428.5 Hz	50000 Hz	0.9%	14.7%	15%	0.3%
PA7 LIN_U1	50429.6 Hz	50000 Hz	0.9%	84.4%	85%	0.6%
PC1 HIN_V1	50428.5 Hz	50000 Hz	0.9%	14.7%	15%	0.3%
PB0 LIN_V1	50429.6 Hz	50000 Hz	0.9%	84.4%	85%	0.6%
PC2 HIN_W1	50428.5 Hz	50000 Hz	0.9%	14.7%	15%	0.3%
PB1 LIN_W1	50429.6 Hz	50000 Hz	0.9%	84.4%	85%	0.6%

PWM frequency: ARR=99, Duty=15%, TCLK=10 MHz, centre-aligned mode (count up-down). Calculation: $F_PWM = TCLK / (ARR+1) / 2 = 10e6 / 100 / 2 = 50,000 \text{ Hz}$. The measured frequency of 50,428.5 Hz deviates by 0.9%. This deviation is entirely explained by the Saleae sampling quantization at 24 MHz: the sample step is $1/24e6 = 41.67 \text{ ns}$. A PWM period of 20 microseconds contains 480 samples; measuring by rising edges, a one-sample error per transition gives approximately $2 * 41.67 \text{ ns} / 20,000 \text{ ns} = 0.42\%$ per period, and with two transitions per period the expected error is 0.5-1.0%, fully consistent with the observed 0.9%. The duty cycle of 15% also matches expectations with a 0.3% error, well within Saleae quantization limits. Channel complementarity is confirmed: HIN channels show 15%, LIN channels show 85% (inverted), matching the CCER configuration (CCxE and CCxNE bits set).

Conclusion: PWM generation is correct. Deviations of 0.9% in frequency and 0.3-0.6% in duty cycle are artifacts of Saleae quantization at 24 MHz, not timer configuration errors.

2.3. Dead-Time Measurement

Dead-time measurements showed unexpected results: all three phases exhibit values around 94-96 ns, approximately 17 times less than the expected value.

Phase	dt_rise	dt_fall	BDTR Reg	DTG bits	Expected DT
U	94 ns	96 ns	0x00000010	16 ticks	~1600 ns
V	95 ns	95 ns	0x00000010	16 ticks	~1600 ns
W	96 ns	94 ns	0x00000010	16 ticks	~1600 ns

BDTR=0x00000010 means DTG[7:0]=0x10 (16 decimal). Per the STM32G4xx Reference Manual, DTG=16 falls in the range 0x00-0x7F, where dead-time is calculated as $DT = DTG * TDTG$. For TIM1: TDTG =

TCLK = 100 ns (at 10 MHz). Thus expected dead-time = $16 * 100 \text{ ns} = 1,600 \text{ ns}$ (1.6 microseconds). However, measured values are only ~95 ns, which is ~17x less.

Two critical factors must be considered when analyzing this discrepancy. First, the centre-aligned mode (CMS=11). In this mode, the timer counts up to ARR, then counts down to zero. The Update event fires at both ARR and zero (double frequency transitions). Dead-time is inserted hardware-wise at each switching event (front-to-back and back-to-front), and must be measured on each edge separately. The measure_deadtime algorithm in SaleaeHelper searches for the interval between a falling edge on one channel and the nearest rising edge on the complementary channel. This is a correct approach for edge-aligned PWM, but in centre-aligned mode, the asymmetric edge behavior may cause incorrect measurements.

Second, Saleae quantization at 24 MHz. The 41.67 ns step means the minimum resolvable interval is one sample (42 ns) or two samples (84 ns). Measured ~95 ns corresponds to approximately 2-3 Saleae samples, near the resolution limit. However, if the actual dead-time were 1,600 ns, it would occupy ~38 samples at 24 MHz, which is easily measurable. Therefore, the observed 95 ns cannot be explained by quantization alone, and the problem lies elsewhere.

Critical finding: measured 95 ns does not match expected 1,600 ns (BDTR=16, TCLK=10 MHz). Most likely cause: incorrect measure_deadtime algorithm in centre-aligned mode, or TIM1 configuration issue (MOE/AOE/OSSR bits). Oscilloscope verification is recommended.

2.4. ADC Calibration

ADC calibration results: offset_i1=2551, offset_i2=2559, offset_in=2555. All three values are close to the midpoint of the 12-bit ADC range (2047-2048), consistent with expected behavior when shunt resistors are disconnected (pins not connected to STEVAL-IPM20B, floating inputs). The deviation from ideal midpoint (2551-2559 vs. 2048) of approximately 500 units can be explained by parasitic voltages on unconnected pins or multiplexer effects.

At zero current (pins not connected to STEVAL), injected ADC conversions are still triggered by TIM1 Update events. If CEN=0 (counter stopped), no Update event is generated, and ADC registers contain stale/garbage values that are interpreted by the GUI as current readings (reported as -13A / -16A). This is not a firmware bug but expected behavior without hardware connection and with TIM1 stopped. When TIM1 is running with PWM output, the injected ADC will sample at the correct phase (triggered by Update), and calibration offsets will correctly subtract the zero-current baseline.

3. Identified Defects

Analysis of the nucleo_debug_tool.py code and test results revealed several defects at various severity levels. These are classified below by severity and area of occurrence.

3.1. Critical Defects

3.1.1. Missing _export_csv Method in SaleaeHelper

At line 240, SaleaeHelper.measure_voltage calls self._export_csv(capture, True, analog_channel, tmp_dir), but the method _export_csv is never defined in the class. Only _export_digital_csv and _read_csv exist. This

means any attempt to measure analog voltage via Saleae (ADC Capture or FOC Capture buttons) will raise an `AttributeError` and completely fail the analog capture function. The digital capture portion (`measure_freq`, `measure_duty`, `measure_deadtime`) works correctly because it uses `_export_digital_csv`. This defect renders the "Capture ADC Waveform" and "Capture FOC Waveforms" analog measurement portions non-functional.

3.1.2. TIMER_CLK / `tlck` Inconsistency

PWMTab defines `TIMER_CLK = 1,000,000` (line 321) but never uses it. Instead, `_update_freq` uses `getattr(root, "tlck", 10,300,000)` (line 419) with a fallback of 10.3 MHz, while `_auto_test_all` uses `getattr(root, "tlck", 10,000,000)` (line 477) with a different fallback of 10.0 MHz. The `root.tlck` attribute is never set anywhere in the code (not in `__init__`, not during SYS telemetry processing). This causes two problems: first, the frequency calculation in the GUI shows an incorrect value (10.3 MHz instead of the actual 10 MHz); second, Auto Test uses the correct fallback of 10 MHz, but only by coincidence. Recommendation: remove `TIMER_CLK`, set `root.tlck = 10,000,000` when processing SYS telemetry, and use a uniform `getattr(root, "tlck", 10,000,000)` everywhere.

3.1.3. Dead-Time Mismatch: 95 ns vs. 1,600 ns

As described in Section 2.3, the observed dead-time of ~95 ns with `BDTR=16` and `TCLK=10 MHz` contradicts the expected 1,600 ns. Possible causes requiring firmware investigation: (1) `OSSR` and `AOE` bits in `BDTR` may alter dead-time generation behavior; (2) `MOE` (Main Output Enable) state may affect the phase relationship of dead-time insertion; (3) centre-aligned mode (`CMS=11`) may create two dead-time phases per period, and the algorithm may measure only one; (4) the `measure_deadtime` algorithm uses `_deadtime_gaps`, which searches for a falling edge on one channel and the nearest rising edge on the other, but in centre-aligned PWM with coupled generation, channels switch in a specific sequence that may not match the algorithm assumption. Oscilloscope verification is essential before connecting power hardware.

3.2. Medium Severity Defects

3.2.1. Bare except Clauses

The code contains "except: pass" constructs (without exception type specification) at lines 455, 470, 523, 542, 670, 741, and others. This makes debugging difficult because any exception (including `KeyboardInterrupt`, `SystemExit`, `MemoryError`) is silently suppressed. Recommendation: replace with "except Exception: pass" at minimum, and add logging at critical points to avoid losing important diagnostic information.

3.2.2. Race Condition in `_tr_cache`

The `self._tr_cache` dictionary in `SaleaeHelper` (lines 57, 188) can be concurrently modified from different threads when concurrent calls to `measure_freq`/`measure_duty`/`measure_deadtime` are made from the same capture. Although this is unlikely in the current single-tab workflow, the architecture allows it (e.g., if multiple capture operations overlap). Recommendation: add a `threading.Lock` for `_tr_cache` access.

3.2.3. Hardcoded Saleae Parameters

`DEVICE_ID`, `GRPC_PORT`, `CACHE_TTL`, and even `NUM_CHANNELS=6` (in `get_transitions`) are hardcoded as global constants. This makes adaptation to different equipment or configurations unnecessarily difficult. Recommendation: move to a configuration file or command-line arguments with sensible defaults.

3.3. Low-Severity Issues

3.3.1. Debug Button in Production Code

SaleaeConnectFrame contains a "Direct" button (`_probe_direct`) for synchronous probe that blocks the GUI during execution. This is debug functionality left in production code. Recommendation: remove or hide behind a debug flag.

3.3.2. FOC Data Buffer for CSV Export

In `ADCTab.on_telemetry` (line 670), `data_buffer` is limited to 10,000 entries, which is a correct memory management decision. However, in `FOCTab._save_csv` (lines 792-797), only the last data point (`self.last_data`) is saved, not the full history. For meaningful FOC data CSV export, a buffer similar to `ADCTab` should be added.

3.3.3. Lambda Closure Fixes (Partially Addressed)

In the previous session, some lambda closures in `_auto_test_all` (lines 458-459) and `_measure_dt_worker` (lines 528-529) were fixed using the default-argument trick to prevent late-binding. Lines 458-459 contain literal strings without parameters and are not affected by late-binding. The critical fix was the main `_log_local` chain, now correctly using thread detection and frozen lambda arguments. The except-as-variable bug (Python 3 deletes exception variables after the except block) was also fixed by capturing the message before the lambda at lines 532 and 761.

4. Readiness Assessment for STEVAL-IPM20B

Subsystem	Status	Confidence	Blocker?
PLL / Clock config	PASS	High	No
PWM generation (freq, duty)	PASS	High	No
Complementary channels	PASS	High	No
Dead-time insertion	FAIL	Low	YES
ADC injected trigger	UNTESTED	Medium	No
ADC calibration	PASS (no hw)	Medium	No
FOC algorithm	UNTESTED	Low	No
GUI UART connectivity	PASS	High	No
GUI Saleae integration	PARTIAL	Medium	No

The PLL clocking subsystem and `SYSCLK` configuration at 170 MHz are fully verified and match expected values. PWM signal generation (50 kHz frequency, 15% duty cycle, complementary channels) is confirmed on all six outputs with acceptable error explained by logic analyzer quantization. ADC calibration shows correct zero-offset values near midrange in the absence of hardware connection.

The critical blocker is the dead-time mismatch: measured 95 ns vs. expected 1,600 ns (BDTR=16, TCLK=10 MHz). Without correct dead-time, connecting the STEVAL-IPM20B will cause shoot-through currents through the upper and lower MOSFET switches during transition, potentially destroying the driver transistors or creating a fire hazard. Before connecting the power stage, it is mandatory to: (1) verify dead-time with an oscilloscope on actual TIM1 signals, (2) confirm that MOE, AOE, and OSSR bits in BDTR are configured correctly, (3) verify that the `measure_deadtime` algorithm works correctly in centre-aligned mode, or rewrite it for this mode.

The FOC algorithm has not been tested with a real motor in the current state. The GUI sends commands "1"/"0" (START/STOP FOC), "s=N" (speed), "i=Id,Iq" (current references), "f" (clear fault), but without access to `main.c` it is impossible to evaluate the quality of the FOC implementation itself (Park/Clarke transform, PI controllers, SVPWM). It is strongly recommended to perform preliminary FOC testing using STM32CubeMonitor or similar tools, observing state variables through SWD/DWG before the first motor run. Protective interrupt handling (overcurrent, overtemperature) should also be verified before the first motor spin attempt.

5. Recommendations

5.1. Priority Actions (Before Connecting STEVAL)

- **CRITICAL: Verify dead-time with an oscilloscope.** Connect a scope to pairs PC0/PA7 (phase U) and measure the actual interval between HIN falling edge and LIN rising edge while TIM1 is running. If dead-time is actually 1.6 us, fix the `measure_deadtime` algorithm for centre-aligned mode.
- **CRITICAL: Implement `_export_csv` in SaleaeHelper.** Without this method, analog capture via Saleae is completely non-functional. Add analog channel export similar to `_export_digital_csv`.
- **HIGH: Set `root.tclk` from SYS telemetry.** When receiving the SYS response "@SYS:CLK=170000000:PSC=16:TCLK=10000000...", set `root.tclk=10000000` and remove the unused `TIMER_CLK` constant.
- **HIGH: Replace bare `except` with `except Exception`.** A minimal change that significantly improves code debuggability.

5.2. Medium-Priority Actions

- **Add `threading.Lock` for `_tr_cache`.** Prevent potential race conditions when transition cache is accessed concurrently from multiple threads.
- **Remove debug "Direct" button from `SaleaeConnectFrame`.** Or move to a hidden debug menu accessible only with a keyboard shortcut or environment variable.
- **Add FOC data buffering for CSV export.** Similar to `ADCTab.data_buffer` for meaningful telemetry analysis after test runs.
- **Rewrite `measure_deadtime` for centre-aligned mode.** The current implementation is designed for edge-aligned PWM. For centre-aligned, the algorithm must account for asymmetric edges on up/down counter cycles and may need to measure dead-time separately on the counting-up and counting-down phases.

5.3. Future Improvements

- **Add waveform visualization script for Saleae captures.** The current interface shows only numeric results. Signal graphs would significantly ease timing debug.
- **Implement protective fault validation before first motor run.** Add reading and display of COMP (comparator) status for overcurrent protection, and temperature monitoring through ADC or the MCU internal sensor.
- **Add unit tests for SaleaeHelper.** Unit tests with mock CSV data will verify `measure_freq`, `measure_duty`, and `measure_deadtime` without requiring a physical Saleae.
- **Move Saleae configuration to a file/CLI.** Replace hardcoded `DEVICE_ID` and `GRPC_PORT` with configuration parameters.